CS216 JAVA PROGRAMMING MASTERY

70 Real-World, Requirement-Based Programming Problems

Aligned with the uploaded CS216 (R-24) syllabus and CS253 OOP Lab Cycle-1 / Cycle-2 references

Instructions

- Implement each assignment in Java and follow the named class, field, constructor, and method requirements.
- Use Scanner for interactive console input unless the assignment explicitly requires command-line arguments or a Swing GUI.
- Validate input and test both valid and invalid cases.
- Do not use external frameworks. Use standard Java classes and libraries.
- These are application-building assignments, not DSA exercises.

Stage 1 — Java fundamentals and problem modeling

Problems 1–10 · Unit I

1. Hospital Patient Billing System

Requirements: Create a class Patient with instance variables patientId (int), patientName (String), consultationFee (double), testCount (int), and roomCharge (double). Create a class BillCalculator with methods calculateSubtotal(Patient p), calculateDiscount(double subtotal), calculateTax(double taxableAmount), and generateBill(Patient p). Apply a 10% discount when the subtotal is at least ₹5,000 and a 5% discount when it is at least ₹2,500 but below ₹5,000. Apply 5% tax after discount. In main(), read details for at least three patients using Scanner, create Patient objects, and print an itemized bill showing subtotal, discount, tax, and final amount. Reject negative fees, charges, or test counts.

2. Electricity Consumption Billing

Requirements: Create a class Consumer with consumerNumber (int), consumerName (String), and unitsConsumed (int). Create a class ElectricityBill with methods calculateEnergyCharge(Consumer c), calculateSurcharge(double charge), and displayBill(Consumer c). Calculate energy charges using progressive slabs: first 100 units at ₹1.50/unit, next 100 at ₹2.50/unit, next 300 at ₹4/unit, and remaining units at ₹6/unit. Add a fixed service charge of ₹75 and a 5% surcharge when energy charges exceed ₹2,000. In main(), read consumer details, calculate bills for multiple consumers, and display each slab's charge, service charge, surcharge, and total. Reject negative units.

3. Employee Attendance Payroll

Requirements: Create an Employee class with employeeId, name, basicMonthlySalary, workingDays, daysPresent, paidLeaveDays, and overtimeHours. Create methods calculatePayableDays(), calculateAttendancePercentage(), calculateBasePay(), calculateOvertimePay(), and displayPayroll(). Calculate payable days as daysPresent + paidLeaveDays; calculate base pay proportionally using payable days / workingDays. Pay overtime at 1.5 times the hourly rate, where hourly rate is basicMonthlySalary / (workingDays * 8). Add an attendance bonus of ₹1,000 when attendance is at least 95%. In main(), process at least three employees and print itemized payroll. Validate days and overtime; payable days must not exceed working days.

4. Retail Invoice Generator

Requirements: Create a Product class with productCode (int), name (String), quantity (int), unitPrice (double), and discountEligible (boolean). Create an Invoice class containing an array of Product objects and methods calculateLineTotal(Product p), calculateSubtotal(), calculateDiscount(), and printInvoice(). Give a 10% invoice discount when subtotal is at least ₹10,000, and an additional 5% discount only on eligible product lines if their combined value is at least ₹5,000. In main(), read details for at least five products, store them in an array, and print each line total, subtotal, discounts, and final amount. Do not use collection classes. Reject negative prices and non-positive quantities.

5. University Examination Result Processor

Requirements: Create a StudentResult class with rollNumber (int), studentName (String), subjectNames (String[]), and marks (int[]). Implement calculateTotal(), calculateAverage(), hasPassedAllSubjects(), calculateGrade(), and displayMarkSheet(). A subject is passed only when marks are at least 40; overall grades are A for average >= 80, B for >= 70, C for >= 60, D for >= 50, otherwise E. If any subject is failed, display the overall result as FAIL regardless of average. In main(), read the number of subjects, then each subject name and mark, create the arrays, and print a complete mark sheet. Validate marks from 0 to 100 and use loops for all subject processing.

6. Travel Expense Reimbursement

Requirements: Create a Trip class with employeeName, tripType (String), transportExpense, hotelExpense, mealExpense, and otherExpense. Create a ReimbursementCalculator class with calculateTotalExpense(), calculateApprovedAmount(), and displayClaim(). For domestic trips, reimburse transport fully, hotel up to ₹4,000 per night for at most 3 nights, and meals up to ₹1,000 per day. For international trips, reimburse transport fully, hotel up to ₹8,000 per night, and meals up to ₹2,500 per day. Other expenses are reimbursed up to ₹1,000 per trip. In main(), read trip details including nights and days, use switch or conditional statements for trip type, and print claimed, approved, and non-reimbursable amounts. Reject negative expenses and invalid trip types.

7. Warehouse Stock Reconciliation

Requirements: Create a StockItem class with itemCode, itemName, openingStock, receivedUnits, dispatchedUnits, damagedUnits, and physicalCount. Implement calculateExpectedStock(), calculateVariance(), and displayReconciliation(). Expected stock is openingStock + receivedUnits - dispatchedUnits - damagedUnits; variance is physicalCount - expectedStock. Display SHORTAGE when variance is negative, SURPLUS when positive, and MATCH when zero. In main(), store at least five StockItem objects in an array, read all values, and print a reconciliation report plus total received, dispatched, and damaged units. Use loops and arrays only; do not use collections. Reject negative quantities and transactions that make expected stock negative.

8. Vehicle Rental Cost Estimator

Requirements: Create a Rental class with customerName, vehicleType, rentalDays, insuranceRequired, and additionalDriver. Create methods calculateBaseCharge(), calculateInsurance(), calculateDriverCharge(), calculateLongRentalDiscount(), and printQuotation(). Use daily rates of ₹1,500 for car, ₹700 for motorcycle, and ₹2,500 for van. Insurance costs ₹300/day for a car, ₹150/day for a motorcycle, and ₹500/day for a van. An additional driver costs ₹500 per rental. Give a 10% base-charge discount for rentals of 7 days or more. In main(), read the options and use switch for vehicle type, then print an itemized quotation. Reject unsupported types and rental periods below one day.

9. Scholarship Eligibility Assessment

Requirements: Create a ScholarshipApplicant class with applicantId, name, averageMarks, attendancePercentage, annualFamilyIncome, and category. Create methods isMeritEligible(), isNeedEligible(), isAttendanceEligible(), and displayEligibility(). Merit scholarship requires average marks >= 85; need scholarship requires family income <= ₹300,000 and average marks >= 60; attendance scholarship requires attendance >= 95% and average marks >= 70. Merit award is ₹20,000, need award ₹15,000, and attendance award ₹5,000; eligible applicants may qualify for more than one award, but total award is capped at ₹30,000. In main(), process multiple applicants and display each eligibility decision and total award. Validate all percentages and nonnegative income.

10. Monthly Business Performance Report

Requirements: Create a BusinessReport class containing monthNames (String[]) and monthlySales and monthlyExpenses (double[]). Implement calculateMonthlyProfit(int monthIndex), calculateAnnualSales(), calculateAnnualExpenses(), calculateAverageProfit(), countProfitableMonths(), and displayReport(). Read sales and expenses for all 12 months into arrays. Display monthly profit/loss, annual sales, annual expenses, annual profit, average monthly profit, the month with highest sales, and the month with lowest profit. If multiple months tie, display all tied months. Use primitive arrays and loops, not collections. Reject negative sales and expenses.

Stage 2 — Classes, objects and encapsulation

Problems 11–20 · Unit I

11. Employee Master and Payroll Register

Requirements: Create an Employee class with private fields employeeId (final int), name (String), department (String), basicPay (double), hra, da, pf, incomeTax, grossSalary, and netSalary (double). Provide a no-argument constructor and overloaded constructors for employee ID/name and for all employee details; use this(...) to reuse initialization where suitable. Implement calculateSalary(), setDepartment(String department), and displayDetails(). HRA is 30% of basic pay for basic pay >= ₹20,000, 20% for >= ₹10,000, otherwise 10%; DA is 80%, PF is 12%, income tax is 10% of gross salary above ₹50,000 and 5% for gross salary from ₹30,001 to ₹50,000, and professional tax is ₹200. Use a static employeeCount and display it. In main(), create at least five employees and print complete salary statements.

12. Bank Account Lifecycle Manager

Requirements: Create a BankAccount class with instance fields accountHolderName (String), balance (double), and accountNumber (int). Declare static fields bankName = "National Bank", interestRate = 5.0, and accountCount = 0. Assign account numbers beginning at 1001 using a static counter. Add a static block that prints a welcome message when the class loads. Implement deposit(double amount), withdraw(double amount), calculateAnnualInterest(), displayAccountDetails(), static updateInterestRate(double newRate), and static displayTotalAccounts(). In main(), create at least three accounts, perform deposits and withdrawals, update the common interest rate, display account details, and print total accounts. Reject non-positive deposits/withdrawals and insufficient-funds withdrawals.

13. Library Book Catalog

Requirements: Create a Book class with private fields title, author, isbn, and price. Provide constructors Book(), Book(String title, String author), and Book(String title, String author, String isbn, double price); use this() for constructor chaining. Implement setDetails(String title, String author, String isbn, double price), returning this to allow method chaining, and displayDetails(). Create a Library class with a Book[] array, an addBook(Book b) method, and displayAllBooks(). Demonstrate passing the current object to the library through a registration method in Book that calls library.addBook(this). In main(), create a Library, create books using different constructors and chained calls, add them, and display all books. Reject duplicate ISBNs and invalid prices.

14. Hospital Appointment Records

Requirements: Create Patient, Doctor, and Appointment classes. Patient must contain patientId, name, age, and contactNumber; Doctor must contain doctorId, name, specialty, and consultationFee; Appointment must contain appointmentId, Patient, Doctor, appointmentDateTime (String), and status. Provide constructors and display methods for all classes. Implement book(), reschedule(String newDateTime), cancel(), and calculateFee() in Appointment. Use static counters to assign unique patient, doctor, and appointment IDs. In main(), create at least three patients, two doctors, and multiple appointments; display each appointment with patient and doctor details. Reject negative fees, invalid ages, and attempts to cancel an already cancelled appointment.

15. Electronics Inventory Manager

Requirements: Create a Product class with final productCode, name, category, price, and quantityInStock. Keep fields private. Implement receiveStock(int quantity), sellStock(int quantity), updatePrice(double newPrice), and displayProduct(). Use a static productCount to track objects created and a static companyName shared by all products. Use a constructor to initialize all required fields. In main(), create at least five products, receive and sell stock, update permitted prices, and display inventory and total product count. Prevent productCode changes, negative prices, zero/negative stock transactions, and sales exceeding available stock.

16. Student Enrollment Profile

Requirements: Create a Student class with private final studentId, name, branch, admissionYear, and an array of enrolledSubjects. Provide constructors and getter methods for protected fields. Implement enrollSubject(String subject), dropSubject(String subject), and displayProfile(). Use a static studentCount. Prevent duplicate subject enrollment and limit each student to six subjects. In main(), create at least three students, enroll and drop subjects, and print their profiles and the total registered students. Validate admission year and reject blank names or subject names. Do not expose the internal subject array directly from a getter.

17. Hotel Room Reservation Manager

Requirements: Create Room with roomNumber, roomType, capacity, nightlyRate, and available fields. Create Reservation with reservationId, guestName, guestCount, room, nights, and status. Provide constructors and methods reserve(), cancel(), calculateCost(), and displayReservation(). Room types and sample nightly rates are Standard ₹2,000, Deluxe ₹3,500, and Suite ₹6,000. Apply a 10% discount for stays of five nights or more. Use static counters for rooms and reservations. In main(), create at least five rooms and three reservations, perform booking and cancellation operations, and print costs. Reject guest counts above capacity, invalid nights, and reservations for unavailable rooms.

18. Vehicle Service Center Records

Requirements: Create Vehicle with registrationNumber, ownerName, model, and odometerReading. Create ServiceRecord with serviceId, Vehicle reference, serviceDate, serviceType, partsCost, and laborCost. Implement calculateInvoice(), updateOdometer(int reading), and displayServiceRecord(). Use a static serviceCount to assign service IDs. In main(), create at least three vehicles and two service records for one vehicle; print all records and the service history for each vehicle. Ensure odometer readings never decrease, costs are nonnegative, and a service record references an existing Vehicle object.

19. Employee Address Registry

Requirements: Create an outer Employee class with employeeId, name, designation, and salary. Inside Employee, define a non-static inner class Address with street, city, state, and postalCode fields. Provide constructors or setter methods for employee and address information, plus displayEmployee() and displayAddress(). Demonstrate that Address can access the enclosing Employee's name or ID. In main(), create at least three Employee objects, create an Address object for each using the syntax associated with a non-static inner class, and display complete profiles. Keep fields private and validate salary and required address fields.

20. Training Institute Course Registration

Requirements: Create Course with courseCode (final), title, fee, and active status; Student with studentId (final), name, and registration count; and an inner EnrollmentReceipt class inside Course containing receiptNumber, student reference, and amountPaid. Provide overloaded constructors for Course and Student, using this() where appropriate. Implement enroll(Student s, double amountPaid), deactivateCourse(), and displayReceipt(). Use static counters for courses and receipts, and a final instituteName constant. In main(), create at least three courses and four students, enroll students, and display receipts. Reject enrollment in inactive courses, duplicate student enrollment in the same course, and payments below zero.

Stage 3 — Inheritance, polymorphism, interfaces and packages

Problems 21–30 · Unit II

21. Employee Payroll Using Inheritance

Requirements: Create a base class Employee with protected employeeId, name, and department, a constructor, calculateSalary(), and displayDetails(). Derive PermanentEmployee with basicPay and allowances, ContractEmployee with hoursWorked and hourlyRate, and Intern with stipend. Use super(...) in each subclass constructor. Override calculateSalary() and displayDetails() in all subclasses. In main(), create an Employee[] array containing at least two objects of each subclass and invoke the methods through Employee references. Demonstrate dynamic method dispatch. Do not use instanceof or type codes to choose the salary formula.

22. Logistics Delivery Cost Calculator

Requirements: Define an abstract class Delivery with shipmentId, destination, weightKg, distanceKm, an appropriate constructor, abstract calculateCost(), and displayDetails(). Derive RoadDelivery, AirDelivery, and ExpressDelivery. Road delivery costs ₹8 per kg per km/100; air delivery costs ₹25 per kg plus ₹12 per km; express delivery costs ₹15 per kg plus ₹10 per km and a fixed ₹500 fee. Override calculateCost() and displayDetails() in each subclass. In main(), store objects in a Delivery[] array and print each quotation using base-class references. Reject non-positive weight or distance and demonstrate an abstract class cannot be instantiated directly.

23. Payment and Refund System

Requirements: Define a PaymentMethod interface with processPayment(double amount), refund(double amount), and a transaction-status method. Create CardPayment, UpiPayment, and WalletPayment classes implementing the interface. Each class must store its own transaction reference and successfully processed amount. Card payments charge a 2% processing fee, UPI payments 0.5%, and wallet payments no fee. Implement refunds that cannot exceed the refundable amount. In main(), create objects through PaymentMethod references, process payments, issue partial refunds, and display transaction details. Simulate processing locally; do not connect to real payment services. Reject non-positive amounts.

24. College Person and Staff Management

Requirements: Create a Person class with private name and age fields, a constructor, getters/setters, and displayDetails(). Derive Student with rollNumber and branch, and Faculty with employeeId, department, and basicPay. Use super(...) in child constructors and override displayDetails(). Faculty must implement calculateGrossSalary(), where gross salary is basic pay plus 30% HRA and 80% DA. In main(), create a Person[] array containing Student and Faculty objects and call displayDetails() through each Person reference. Validate age and basic pay. Demonstrate parent constructor calls and dynamic method dispatch.

25. Insurance Premium Calculator

Requirements: Define abstract class InsurancePolicy with policyNumber, policyholderName, coverageAmount, abstract calculateAnnualPremium(), and displayPolicy(). Derive HealthPolicy, VehiclePolicy, and PropertyPolicy. For this exercise, use premiums of 4% of coverage for health, 3% for vehicle, and 1.5% for property; vehicle policy also adds a fixed ₹2,000 fee. Override calculation and display methods. In main(), create a mixed InsurancePolicy[] array and display each premium through base references. Reject non-positive coverage. State in the output that these are classroom sample rates, not real insurance quotations.

26. Banking Products and Loan Eligibility

Requirements: Create a BankProduct base class with productCode, holderName, and balance. Derive SavingsAccount, CurrentAccount, and FixedDeposit. Define a LoanEligibility interface containing assessEligibility(double monthlyIncome) and a minimum-income constant. Implement LoanEligibility only in

product types that support loan assessment. Savings accounts earn 4% annual interest, current accounts charge a ₹500 annual maintenance fee, and fixed deposits calculate interest using a supplied annual rate and term. In main(), create each product type, display balances/interest/fees, and assess eligibility through interface references. Validate rates, terms, balances, and income.

27. University Assessment Framework

Requirements: Define abstract class Assessment with assessmentId, studentName, maximumMarks, abstract evaluate(double marks), and displayResult(). Create InternalAssessment, LaboratoryAssessment, and FinalAssessment subclasses with their own pass thresholds (40%, 50%, and 40% of maximum marks respectively). Override evaluate() and displayResult(). Override toString() to return a readable assessment summary and equals()/hashCode() based on assessmentId. In main(), create mixed Assessment objects, evaluate supplied marks through base references, and display results. Reject marks outside 0..maximumMarks and duplicate assessment IDs in the test data.

28. Vehicle Rental and Maintenance

Requirements: Create an abstract Vehicle class with registrationNumber, model, passengerCapacity, and available status. Define Rentable with calculateRentalCost(int days), rent(), and returnVehicle(); define Maintainable with recordMaintenance(String description) and displayMaintenanceHistory(). Implement Car, Motorcycle, and Van, with daily rates ₹1,500, ₹700, and ₹2,500. Each class must implement Rentable; each may implement Maintainable. In main(), store vehicles in a Vehicle[] array, generate quotations through base references, rent and return vehicles, and demonstrate maintenance through Maintainable references. Reject invalid rental periods and unavailable vehicles.

29. College Management Package

Requirements: Create a package named college containing Person, Student, and Faculty classes. Person must have private name and age fields and public methods to set and display them. Student extends Person and adds rollNumber and branch. Faculty extends Person and adds employeeId, department, and basicPay, plus calculateGrossSalary(). In a separate package named app, create CollegeDemo, import college classes, instantiate Student and Faculty, assign them to Person references, and call display methods. Compile and run the program using package folders and an appropriate classpath. Demonstrate that private fields are accessed through public methods rather than directly from app.

30. Vehicle Specification Interfaces

Requirements: Define interface Vehicle with constant SPEED_LIMIT and method displayDetails(); define Engine with engineType(); define CarFeatures extending both and adding carModel() and fuelType(). Implement CarFeatures in PetrolCar and ElectricCar. Each class must store model, manufacturer, and relevant engine/fuel information, and implement all inherited methods. In main(), create both objects, call methods through CarFeatures references, and display interface constants. Do not duplicate the common method contract in unrelated classes. Use constructors to initialize required attributes.

Stage 4 — Strings, command-line arguments and exception handling

Problems 31–38 · Units II–III

31. Customer Support Ticket Text Processor

Requirements: Create a SupportTicket class with ticketId, customerName, and description. Implement countWords(), countCharacters(), countSentences(), normalizeWhitespace(), containsKeyword(String keyword), and displaySummary(). Use String methods including trim(), split(), length(), contains(), equalsIgnoreCase(), and substring(). In main(), read a complaint, process it, and display word/character/sentence counts, a normalized preview of at most 100 characters, and whether selected keywords occur. Handle blank input and repeated spaces. Do not use external text-processing libraries.

32. Confidential Document Text Editor

Requirements: Create a TextDocument class containing a StringBuffer field. Implement appendText(String text), insertText(int position, String text), deleteText(int start, int end), replaceText(int start, int end, String replacement), and displayText(). In main(), initialize a document and present a menu for append, insert, delete, replace, and display operations; display the text after every modification. Use StringBuffer for every modification. Validate indices and handle invalid operations without terminating the menu loop. Do not use String concatenation to perform edits.

33. Command-Line Student Result Analyzer

Requirements: Implement calculate(String studentName, int... marks), which calculates total, average, highest mark, and count of passing subjects (pass mark 40). In main(), read the student's name from args[0] and parse remaining arguments as marks. If no marks are supplied, print exactly "No marks entered". If no student name is supplied, display usage instructions. Catch NumberFormatException for malformed marks and validate each mark from 0 to 100. Run the program with multiple command-line arguments; do not replace command-line input with Scanner.

34. Employee ID and Email Validation

Requirements: Create an EmployeeRegistration class with employeeId, name, and email. Implement isValidEmployeeId(), isValidEmail(), normalizeName(), and displayValidationReport(). Employee IDs must match the format EMP followed by exactly four digits. For this exercise, email must contain one @, a nonempty local part, a domain containing a dot, and no spaces. In main(), process multiple registration entries and report every invalid field for each entry. Use String methods and character checks, not external validation libraries or regular-expression frameworks. Clearly treat these checks as basic format validation only.

35. Invoice Reference Generator

Requirements: Create an InvoiceReference class with departmentCode, year, and sequenceNumber. Implement generateReference() using StringBuffer, parseReference(String reference), and displayComponents(). Format references as DEPT-YYYY-NNNNN, where NNNNN is a five-digit sequence. In main(), accept an optional starting sequence from command-line arguments; if absent, use 1. Generate at least five consecutive references, then accept sample references and validate/extract their components. Reject invalid department codes, years, and sequence numbers. Handle malformed command-line arguments without crashing.

36. Exception-Safe ATM Simulator

Requirements: Create BankAccount with accountNumber, pin, and balance. Implement deposit(double amount), withdraw(double amount), and validatePin(String enteredPin). Define custom exceptions

InsufficientFundsException, InvalidAmountException, and AuthenticationException. Use try, multiple catch clauses, throw, throws, and finally in transaction processing. After three failed PIN attempts, reject further transactions in the current session. In main(), provide a menu for balance inquiry, deposit, and withdrawal. A failed operation must leave the balance unchanged, and finally must print a transaction-completion message.

37. Hotel Booking Validation

Requirements: Create Room and Reservation classes. Define custom exceptions InvalidDateRangeException, RoomUnavailableException, and OccupancyLimitException. Implement validateDates(), validateGuests(), and confirmReservation() with appropriate throws declarations. Use a nested try block to separate input/date validation from booking confirmation; use multiple catch clauses for the custom exceptions and finally to report completion. In main(), attempt valid and invalid reservations and show that failed reservations do not change room availability. Check that checkout follows check-in and guest count does not exceed room capacity.

38. Transaction Record Import and Audit

Requirements: Create TransactionRecord with transactionId, accountNumber, transactionType, and amount. Implement parseRecord(String line), validate(), and applyTo(BankAccount account). Define InvalidTransactionRecordException. Each input line uses the format ID,ACCOUNT,TYPE,AMOUNT separated by commas. Use try/catch for numeric parsing and nested try blocks to distinguish parsing failures from transaction failures. Use finally to count every processed record. In main(), process a batch containing valid and malformed lines, then display accepted/rejected counts and account balances. A rejected record must not partially change an account.

Stage 5 — Multithreading, synchronization and communication

Problems 39–48 · Unit III

39. Railway Seat Reservation Simulator

Requirements: Create a Coach class with totalSeats and an array indicating booked seats. Implement synchronized bookSeat(int seatNumber, String passengerName) and displayAvailability(). Create one booking task by extending Thread and another by implementing Runnable; each task attempts to reserve a specified seat. In main(), start at least five concurrent booking tasks, join them, and print final seat allocation. Run a controlled comparison with an unsynchronized method to demonstrate the risk of race conditions. Ensure the synchronized version never allocates one seat to two passengers.

40. Concurrent Bank Transfers

Requirements: Create BankAccount with accountNumber, balance, deposit(), and synchronized withdrawal(). Create a TransferTask implementing Runnable with source account, destination account, and amount. Implement transfer() so both accounts are updated consistently, using a documented lock-acquisition strategy to avoid deadlock. In main(), create at least three accounts and launch multiple transfers concurrently, then join all threads and display balances. Verify that total money remains constant except for explicitly recorded fees. Reject invalid amounts and transfers with insufficient funds.

41. Food Delivery Order Workflow

Requirements: Create FoodOrder with orderId, status, and methods placeOrder(), markPrepared(), assignDelivery(), and markDelivered(). Create separate Runnable tasks for restaurant preparation and delivery assignment. Use a synchronized state transition or wait()/notifyAll() so delivery cannot begin before preparation is complete. In main(), process at least three orders concurrently and print every state transition. Reject invalid transitions such as delivering a cancelled order. Do not use Thread.sleep() as the sole mechanism for coordinating task order.

42. Warehouse Producer–Consumer System

Requirements: Create a Warehouse class with currentStock and capacity. Implement synchronized produce(int quantity) and consume(int quantity), using wait() while the warehouse is full or insufficient stock is available, and notifyAll() after stock changes. Create Supplier and DispatchWorker classes implementing Runnable. In main(), start at least two suppliers and two dispatch workers, run a fixed number of operations, and display inventory changes. Use while loops around wait conditions and handle InterruptedException correctly. Stock must remain between zero and capacity.

43. Hospital Department Task Scheduler

Requirements: Create HospitalTask extending Thread with taskName and duration/step count. Override run() to report progress and completion. Create tasks for laboratory, pharmacy, and administration. In main(), start the threads, report their states at meaningful checkpoints, join required tasks before generating a final report, and demonstrate interruption of one optional task. Catch InterruptedException and restore the interrupt status when appropriate. Do not assume a fixed execution order or rely on exact timing for correctness.

44. Deadlock Demonstration and Prevention

Requirements: Create two Resource objects and two Runnable tasks. In a demonstration method, make each task acquire one resource and then request the other using synchronized blocks, documenting the conditions that can cause deadlock. Provide a separate corrected method that always acquires resources in a consistent order. In main(), run the safe version to completion and isolate the deadlock demonstration so it cannot prevent

the program from exiting indefinitely. Print which strategy is being demonstrated and the observed completion status. Do not make the entire application depend on an unbounded deadlock.

45. Cinema Ticket Counter

Requirements: Create Screening with screeningId, totalSeats, and booked-seat information. Implement synchronized bookSeat(), cancelSeat(), and displayAvailability(). Create BookingClerk tasks implementing Runnable; each receives a list of seat requests. In main(), run several clerks concurrently, then print confirmed tickets, rejected requests, and remaining seats. A cancellation must succeed only for a currently booked seat. Test repeated requests for the same seat and verify that no seat is double-sold.

46. Online Order Fulfillment Pipeline

Requirements: Create Order with orderId, status, and inventoryReserved flag. Implement validateOrder(), reserveInventory(), packOrder(), and dispatchOrder() with explicit permitted state transitions. Create Runnable tasks for processing different orders concurrently. Use synchronization for shared inventory and coordination for dependent steps. In main(), submit at least five orders, including one invalid order and one with unavailable stock, and print each final state. Dispatch must occur only after successful validation, reservation, and packing.

47. Shared College Printing Service

Requirements: Create PrintJob with jobId, department, documentName, and pageCount. Create Printer with a synchronized printDocument(PrintJob job) method that allows one job at a time and tracks completed pages. Create request threads that submit jobs and support cancellation only before processing begins. In main(), submit at least six jobs from three departments, wait for all threads to finish, and display job status and department-wise page totals. Reject non-positive page counts and ensure cancelled jobs are not marked completed.

48. Inter-Department Purchase Approval

Requirements: Create PurchaseRequest with requestId, department, amount, and status. Create DepartmentReviewer and FinanceReviewer Runnable tasks. A finance review may begin only after department approval; rejection or cancellation must stop the workflow. Use synchronized state methods and wait()/notifyAll() or another explicit thread-coordination mechanism. In main(), process multiple requests concurrently and display each approval history. Ensure that rejected requests never reach the payment-approved state and that all worker threads can terminate cleanly.

Stage 6 — Generics and collections

Problems 49–58 · Unit III

49. Generic College Record Repository

Requirements: Implement a generic class Repository<T> backed by an appropriate collection. Provide add(T item), getById(String id), update(String id, T item), remove(String id), and displayAll(). Define a Record interface with getId(), then implement it in Student, Faculty, and Course classes. Implement a generic method to display records from a Repository<? extends Record>. In main(), create separate repositories for students, faculty, and courses; add, retrieve, update, remove, and display records. Avoid raw types and unchecked casts.

50. Hospital Patient Registration Desk

Requirements: Create Patient with patientId, name, age, and status. Create PatientRegistry using a Map<Integer, Patient> to retrieve patients by unique ID. Implement registerPatient(), updatePatientStatus(), removePatient(), findPatient(), and displayAllPatients(). Use an Iterator to display records and count active versus discharged patients. In main(), register at least five patients, update statuses, attempt a duplicate ID, and search for an absent ID. Report invalid operations without terminating the application.

51. Retail Product Catalog

Requirements: Create Product with productId, name, category, price, and stock. Create InventoryManager using Map<String, Product>. Implement addProduct(), updateStock(), updatePrice(), removeProduct(), findProduct(), and displayLowStock(int threshold). Use an Iterator or entrySet iteration for inventory reports. In main(), add at least six products, update stock and prices, remove a product, and print low-stock items. Reject duplicate IDs, negative prices, and stock reductions below zero. Do not implement a custom map or sorting algorithm.

52. University Course Registration Portal

Requirements: Create Course with courseCode, title, capacity, and a Set<String> of enrolled student IDs. Create RegistrationService with registerStudent(), dropStudent(), and displayEnrollment(). Maintain courses in a Map<String, Course>. Prevent duplicate enrollment and enforce capacity. In main(), create at least three courses and register several students, including duplicate and full-course attempts; then drop one student and display updated enrollments. Use collection interfaces and suitable implementations such as HashMap and HashSet. Do not implement custom collection structures.

53. Shopping Cart and Order History

Requirements: Create Product with productId, name, and unitPrice; CartItem with Product and quantity; ShoppingCart with a Map<String, CartItem>; and Order with an immutable snapshot of purchased items. Implement addItem(), updateQuantity(), removeItem(), calculateSubtotal(), checkout(), and displayInvoice(). Apply a 10% discount when subtotal is at least ₹5,000. In main(), add and modify cart items, checkout, then change the active cart and verify that the completed order remains unchanged. Validate quantities and prices. Use standard collection classes.

54. Generic Notification Queue

Requirements: Define a Notification interface with getRecipient() and deliver(). Implement EmailNotification, SmsNotification, and AppNotification. Create a generic NotificationQueue<T extends Notification> backed by a Queue<T>, with enqueue(), processNext(), and displayPending(). Create a generic method that counts records satisfying a supplied status rule or reports pending items. In main(), enqueue notifications of different implementations through an appropriate common type, process them, simulate failures, and display pending/failed counts. Do not connect to external messaging services; do not mark a failed delivery successful.

55. Hospital Appointment and Waiting List

Requirements: Create Appointment with appointmentId, patientId, doctorId, and timeSlot. Create AppointmentManager using a Map<String, Appointment> for confirmed appointments and a Queue<AppointmentRequest> for each waiting list. Implement book(), cancel(), reschedule(), and displaySchedule(). Prevent overlapping appointments for the same doctor and promote the next valid waiting request when a slot becomes free. In main(), test full slots, cancellations, duplicate requests, and rescheduling. Use standard collections and validate identifiers and time-slot values.

56. Bank Transaction Ledger

Requirements: Create Transaction with transactionId, accountNumber, type, amount, and referenceTransactionId for reversals. Create TransactionLedger with a List<Transaction> and a Map<String, BankAccount>. Implement recordTransaction(), reverseTransaction(), displayAccountHistory(), and generateAuditSummary(). Use an Iterator to process transaction records and calculate deposits, withdrawals, and reversals by account. A transaction ID must be unique; a reversal must reference an existing unreversed transaction and must not delete the original. In main(), create multiple accounts, process transactions, and print an audit report.

57. Generic Examination Report Generator

Requirements: Create a generic AssessmentResult<T extends Number> class containing studentId, subjectName, marks, and maximumMarks. Implement validation and percentage calculation. Create a ResultManager that stores results in suitable Map/List collections and provides a generic report method for different numeric mark representations where valid. In main(), enter results for multiple students and subjects, display student-wise and subject-wise reports, and identify missing assessments explicitly. Validate that marks lie between zero and maximum marks. Do not replace missing marks with zero unless clearly marked as a chosen policy.

58. College Event Registration Manager

Requirements: Create Event with eventId, title, capacity, a Set<String> of registered student IDs, and a Set<String> of attendees. Create EventManager using Map<String, Event>. Implement register(), cancelRegistration(), markAttendance(), and displayEventReport(). Prevent duplicate registration, reject registration after capacity is reached, and allow attendance only for registered participants. In main(), create multiple events, register and cancel participants, mark attendance, and display registration and attendance counts. Use collection interfaces and iterators where appropriate.

Stage 7 — Event handling, Swing and integration

Problems 59–70 · Unit IV

59. Employee Registration Swing Application

Requirements: Create an Employee class with employeeId, name, department, and salary. Create EmployeeForm extending JFrame with JTextField components for ID, name, and salary; a JComboBox for department; and Add, Clear, and Display buttons. Register ActionListener handlers for each button. Store accepted employees in an ArrayList<Employee> and display records in a JTable. Validate mandatory fields, numeric ID, and nonnegative salary; show errors using JOptionPane. In main(), launch the GUI using SwingUtilities.invokeLater(). Invalid records must not be added.

60. Student Result Calculator GUI

Requirements: Create StudentResult with studentName and an array of marks, plus calculateTotal(), calculateAverage(), and calculateResult(). Create a JFrame containing name and marks text fields, Calculate, Reset, and Exit buttons, and labels for total, average, and pass/fail result. Attach ActionListener objects to the buttons. Validate that all marks are integers from 0 to 100 and that every subject meets the 40-mark pass threshold. Display validation errors with JOptionPane. Keep calculation logic in StudentResult rather than inside the listener.

61. Bank Account Operations GUI

Requirements: Create BankAccount with accountNumber, holderName, balance, deposit(double), withdraw(double), and getBalance(). Create a JFrame with account details, an amount JTextField, Deposit, Withdraw, and Balance buttons, plus a JTextArea inside JScrollPane for transaction history. Attach listeners that invoke the BankAccount methods and update the displayed balance. Reject malformed amounts, non-positive transactions, and insufficient funds using JOptionPane. Ensure the GUI displays the same balance as the account object after each operation.

62. Hospital Registration and Appointment GUI

Requirements: Create Patient, Doctor, and Appointment classes. Build a JFrame with patient text fields, a department JComboBox, a doctor JComboBox, appointment-time input, and Book, Cancel, and Clear buttons. Use ActionListener handlers to validate fields and invoke appointment operations. Display confirmed appointments in a JTable within a JScrollPane. Reject missing patient data, unavailable doctors, duplicate appointments, and invalid time values. Keep validation and booking logic in separate classes. Update the table only after a successful operation.

63. Shopping Cart Desktop Application

Requirements: Create Product, CartItem, and ShoppingCart classes. Build a JFrame with product selection through a JComboBox, quantity input through a JTextField, and Add, Remove, Calculate Total, and Checkout buttons. Display cart entries in a JTable. Implement ActionListener handlers that update the ShoppingCart model and refresh the table. Apply a 10% discount when subtotal reaches ₹5,000 and show subtotal, discount, and final amount. Validate product selection and quantity; completed orders must retain a snapshot of their items.

64. Hotel Room Booking Interface

Requirements: Create Room and Reservation classes. Build a JFrame containing guest-name input, room-type JComboBox, guest-count field, check-in and check-out input fields, and Check Availability, Book, Cancel, and Reset buttons. Display active reservations in a JTable. Implement listeners that validate date order, room capacity, and availability before confirming a booking. Calculate cost from room rate multiplied by nights and apply a 10% discount for five or more nights. A failed booking must not change room status.

65. College Administration with Tabbed Panes

Requirements: Create Student, Faculty, and Course classes. Create an AdministrationFrame extending JFrame with a JTabbedPane containing Student Management, Faculty Management, and Course Management panels. Each panel must contain input fields, Add and Clear buttons, and a JTable for records. Use ActionListener handlers and separate model/service classes for validation and updates. Validate unique IDs and required fields. Adding a record in one tab must not erase or duplicate records in another tab. Display validation feedback through JOptionPane.

66. Library Catalog with JTable

Requirements: Create Book with isbn, title, author, and availability. Create LibraryCatalog with addBook(), updateAvailability(), findBook(), and getBooks(). Build a JFrame with text fields, Add Book and Loan/Return buttons, and a JTable inside JScrollPane. Use a table model or refresh method to reflect changes in the underlying catalog. Allow a selected row to be used for a loan or return operation. Reject duplicate ISBNs and loans of unavailable books. Display book details and operation status clearly.

67. College Organization Directory

Requirements: Create Department and DirectoryPerson classes. Build a JFrame containing a JTree whose root is the college and whose child nodes represent departments, faculty, and subgroups. Use a tree model such as DefaultMutableTreeNode/DefaultTreeModel. Add a details panel that updates when a node is selected using a TreeSelectionListener. Provide controls to add a department or directory person under a valid parent node. Validate names and prevent duplicate IDs. The selected node's details must correspond to the underlying record.

68. Customer Support Case Manager

Requirements: Create SupportCase with caseId, customerName, description, priority, and status. Create CaseManager with createCase(), updateCase(), closeCase(), and findCase(). Build a JFrame with text fields, a priority JComboBox, a status JComboBox, a multiline JTextArea in JScrollPane, and Create, Update, and Close buttons. Display cases in a JTable and show the selected case's full details. Register listeners for all actions. Reject duplicate case IDs and blank descriptions. Closed cases cannot be modified unless explicitly reopened by a separate permitted operation.

69. Event-Driven Text Editor

Requirements: Create TextDocument containing a StringBuffer and methods appendText(), insertText(), deleteText(), replaceText(), countWords(), and countCharacters(). Create a JFrame with a JTextArea in JScrollPane and buttons for Append, Insert, Delete, Replace, Word Count, and Character Count. Use ActionListener handlers and selected text/selection positions for editing. Display validation errors for invalid positions and handle empty content. Use StringBuffer for all document modifications. The text area must reflect each successful modification immediately.

70. Integrated College Management Desktop Application

Requirements: Create packages college.model, college.service, college.exception, and college.ui. In the model package, implement Person, Student, Faculty, Course, Enrollment, and AssessmentResult classes using encapsulation, constructors, inheritance, overriding, and suitable interfaces. In service, implement generic Repository<T extends Identifiable>, registration/enrollment operations, and collection-backed storage using Map, List, and Set. Define custom exceptions for duplicate IDs, invalid registration, and failed enrollment. In ui, create a Swing JFrame with JTabbedPane panels for students, faculty, courses, and results; use text fields, combo boxes, buttons, tables, and event listeners. Use a background thread for a simulated report-generation task and synchronize shared data where needed. In main(), launch the GUI on the Swing event-dispatch thread. Validate input, preserve data consistency, and demonstrate all specified concepts in a working application.

Syllabus Coverage Notes

The problem set covers the practical programming areas of Units I–IV: fundamentals; classes, constructors, static, access control, this/final and inner classes; inheritance, abstract classes, interfaces and packages; strings and command-line arguments; exception handling; multithreading, synchronization, inter-thread communication and deadlock; generics and collections; event handling and Swing components.

Some syllabus items are primarily conceptual or legacy topics and are not ideal as standalone modern application assignments: Java history/evolution and buzzwords, garbage collection/finalize(), and JApplet. Review these alongside the relevant stages. finalize() and JApplet are legacy APIs; practical use depends on the JDK/lab environment.

The two lab reference documents informed the requirement-driven style, including explicit class definitions, instance/static variables, method names, calculations, constructor behavior, main-method tasks, and expected demonstrations.